

Report
Of
STUDY CONTENT GENERATOR

Virtual Internship at YBI Foundation
Domain: Artificial Intelligence and Generative AI

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By
Roll Number of Student: CS-2341055
Name of Student: NAVNEET KUMAR
Section: 4CSE3
Batch: 2023-27

2026-27

CANDIDATE'S DECLARATION

I, **NAVNEET KUMAR** do hereby declare that the internship report titled "**Study Content Generator**" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Navneet Kumar

Roll No.: CS-2341055

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I thank YBI Foundation for giving me the opportunity to undergo the one-month virtual internship in the domain of Artificial Intelligence and Generative AI, and the mentors and trainers of the programme for their guidance during the sessions and while I was building the project "Study Content Generator".

I also thank the faculty members of the School of Computer Science and Engineering, IILM University, Greater Noida, for their encouragement and support throughout the internship and the preparation of this report.

I express my gratitude to my parents for their blessings.

Signature

Student Name: Navneet Kumar

Roll No.: CS-2341055

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	-----
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
	List of Tables	iv
	List of Figures	iv
3.	Internship Completion Certificate	-----
4.	Project Description	1
4.1	Introduction	1
4.2	Organization Profile	1
4.3	Problem Statement	2
4.4	Project Objectives	2
4.5	Scope of the Project	3
4.6	Technologies and Tools Used	3
4.7	System Architecture	4
4.8	Methodology	6
4.9	Expected Outcomes	9
4.10	Certificates of Completion and Other Communication Mail Proof	10
5.	Bibliography / References	11

LIST OF TABLES

Table 4.1: Summary of the internship.....	2
Table 4.2: Technologies and tools used in the project.....	4
Table 4.3: System requirements.....	4
Table 4.4: Output modes of the Study Content Generator.....	6

LIST OF FIGURES

Figure 4.1: System architecture of the Study Content Generator	5
Figure 4.2: Workflow of the application.....	7
Figure 4.3: User interface of the Study Content Generator	8

CERTIFICATE



Corporate Identification Number
U80903DL2020NPL371984

19894000006954239



Ybi Foundation

This is to Certify that

Navneet Kumar

has self motivation and willingness to learn, engage in day to day tasks and projects with high level of professionalism in

Artificial Intelligence and Generative Ai

Internship Period: 1 Month, Completion on: Sunday, Aug 23 2026



Sunday, Aug 23 2026
Dr. Alp Yadav
Navneet Kumar

Scan QR Code

Check Credential Id for Certificate Verification : **QT3G10MBX2IL3**

www.ybifoundation.com (+91) 966 798 7711 support@ybifoundation.com

Self generated by intern

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

Artificial Intelligence (AI) has changed the way information is created, organised and consumed. Within AI, Generative AI, and in particular Large Language Models (LLMs), can read natural-language text and produce new text such as summaries, explanations and answers [1], [2]. These abilities open new opportunities in education, where students regularly face large volumes of notes, chapters and reference material that must be condensed into a form suitable for revision.

This report describes **Study Content Generator**, the project developed during a one-month virtual internship at YBI Foundation in the domain of Artificial Intelligence and Generative AI. The project is a Streamlit web application that accepts educational content in CSV or TXT format and converts it into interactive study material: summaries, question-and-answer study guides and flashcards. Users can also type a custom query and receive an answer based on the uploaded content.

The application follows the Retrieval-Augmented Generation (RAG) approach [3]. The uploaded content is split into chunks, converted into vector embeddings using SentenceTransformers [4], and stored in the ChromaDB vector store. When the user requests an output, the most relevant chunks are retrieved and passed, together with a suitable prompt, to an LLM served locally through Ollama. LangChain coordinates this retrieval and prompting workflow. Because every component runs on the user's own machine, no cloud API is needed and the uploaded content does not leave the device.

The rest of this chapter presents the organization profile, problem statement, objectives, scope, technologies used, system architecture, methodology and expected outcomes of the project.

4.2 Organization Profile

YBI Foundation conducted the virtual internship programme in Artificial Intelligence and Generative AI in which this project was carried out. The internship was completed online over a period of one month and followed a project-based approach: the concepts of AI and Generative AI were introduced first and were then applied by building a working project. Table 4.1 summarises the internship.

Table 4.1: Summary of the internship

Particular	Details
Organization	YBI Foundation
Mode of internship	Virtual (online)
Duration	One month
Domain	Artificial Intelligence and Generative AI
Project title	Study Content Generator
Intern	Navneet Kumar (Roll No. CS-2341055, Section 4CSE3)
Programme	B.Tech CSE, School of Computer Science and Engineering, IILM University, Greater Noida

4.3 Problem Statement

Teachers and students spend a large amount of time turning raw study material into forms that are suited to revision. Preparing a summary, framing practice questions and making flashcards by hand is slow, and the effort grows with every new chapter. Existing AI tools can help, but they bring their own difficulties:

- **Manual effort:** creating varied study resources from the same content is repetitive and time-consuming.
- **Dependence on the cloud:** most AI tools need a constant internet connection, and many need paid API access.
- **Privacy:** teachers' notes and institutional content have to be uploaded to third-party servers.
- **Generic answers:** a language model asked a question without the source material may give answers that are not tied to the content the learner is actually studying.

Problem statement: to design and develop a web-based application that accepts educational content in CSV or TXT format and, using a locally hosted LLM together with vector-based retrieval, generates summaries, question-and-answer study guides, flashcards and custom-query answers grounded in the uploaded content, without relying on any cloud API.

4.4 Project Objectives

The objectives of the project are:

- To build a simple Streamlit interface through which a user can upload CSV or TXT educational content.
- To process and index the uploaded content using text chunking, SentenceTransformers embeddings and the ChromaDB vector store.
- To generate four types of output: summaries for high-school students, Q&A study guides, flashcards and custom-query answers.
- To run the LLM locally through Ollama so that the application works without a cloud API and keeps user content private.
- To use LangChain to connect retrieval, prompt templates and the language model into one workflow.
- To gain practical experience of building a Generative AI application during the internship.

4.5 Scope of the Project

Within the scope of the project:

- Uploading educational content in CSV or TXT format through the sidebar of the application.
- Generating a summary, a Q&A study guide, flashcards or a custom-query answer from the uploaded content.
- Fast retrieval of relevant content using the ChromaDB vector store.
- Local deployment on the user's own computer using Ollama, with no cloud API.

Limitations:

- Only CSV and TXT files are supported; other formats such as PDF, DOCX or images are not handled.
- The application needs Ollama installed and running (ollama serve) on the same machine.
- The quality and speed of the output depend on the local model chosen and on the hardware available.
- Generated material is produced by a language model and should be reviewed by a teacher or student before it is relied upon.

4.6 Technologies and Tools Used

The project is written in Python and combines a web framework, an LLM orchestration library, a local model runtime, a vector database and an embedding library. Table 4.2 lists the technologies and the role each one plays.

Table 4.2: Technologies and tools used in the project

Technology	Category	Role in the project
Python	Programming language	Implements the application logic and connects all the components [10].
Streamlit	Web framework	Provides the user interface: sidebar file upload, output-type selection, generate button and display of results [5].
LangChain	LLM framework	Coordinates document loading, text splitting, retrieval and prompt-based generation [6].
Ollama	Local LLM runtime	Runs the language model on the user's machine and serves it to the application (ollama serve) [7].
ChromaDB	Vector store	Stores the embeddings of the content chunks and supports fast similarity search [8].
SentenceTransformers	Embedding library	Converts text chunks and queries into numerical vectors that capture their meaning [4], [9].

The system requirements for running the application are given in Table 4.3.

Table 4.3: System requirements

Requirement	Details
Programming language	Python 3.9 or higher
LLM runtime	Ollama installed locally, with the server running (ollama serve)
Python packages	Streamlit, LangChain, ChromaDB, SentenceTransformers and the other packages listed in requirements.txt
Input files	Compatible CSV or TXT files containing educational content
Hardware	A computer with enough memory and processing power for the chosen local model
Browser	Any modern web browser to open the Streamlit interface
Internet	Needed only for the one-time installation of packages and models; not needed for generating study material

4.7 System Architecture

The Study Content Generator uses a layered Retrieval-Augmented Generation architecture, shown in Figure 4.1. The user interacts only with the Streamlit interface; the other layers work behind it.

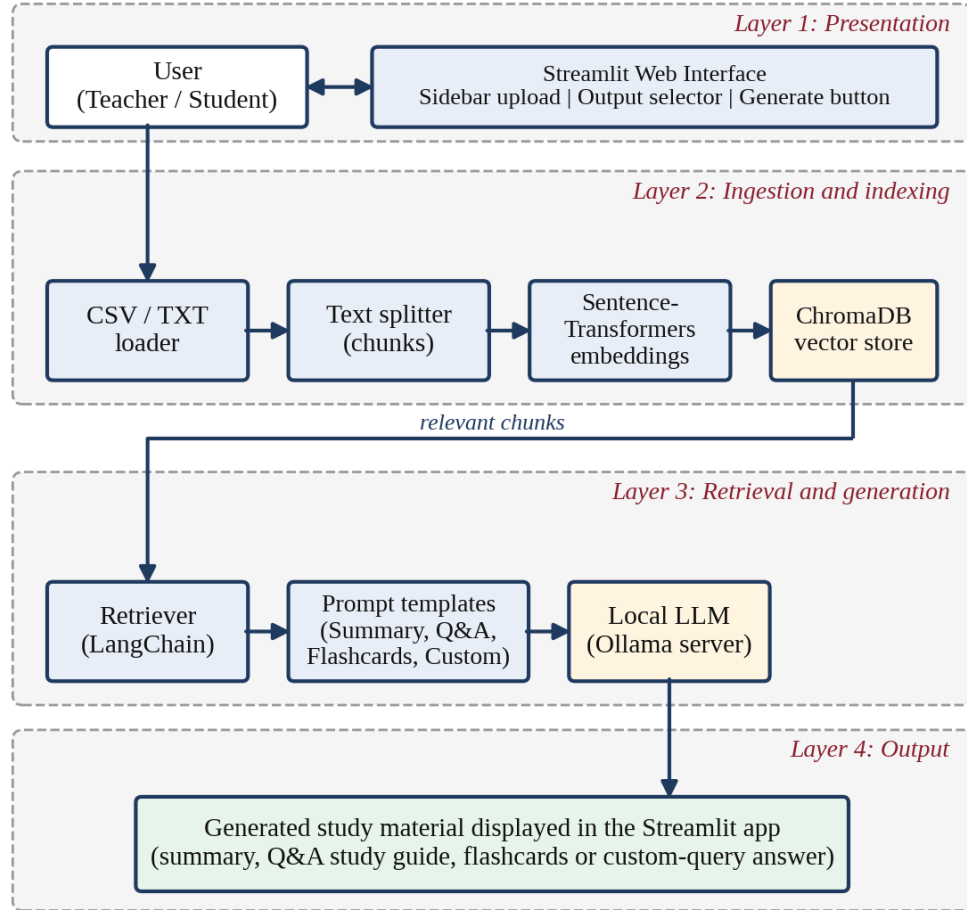


Figure 4.1: System architecture of the Study Content Generator

The four layers of the architecture are described below.

- **Presentation layer:** the Streamlit web interface. The user uploads a CSV or TXT file in the sidebar, chooses the type of output (Summary, Q&A, Flashcards or Custom Query) and clicks the button to generate the material.
- **Ingestion and indexing layer:** the uploaded file is read by a loader and divided into smaller chunks by a text splitter. Each chunk is converted into an embedding with SentenceTransformers and stored in ChromaDB.
- **Retrieval and generation layer:** LangChain retrieves the chunks most relevant to the request from ChromaDB, inserts them into a prompt template chosen for the selected output type, and sends the prompt to the local LLM served by Ollama.

- **Output layer:** the text produced by the model is displayed in the Streamlit application as study material.

The application offers four output modes. They are compared in Table 4.4.

Table 4.4: Output modes of the Study Content Generator

Mode	What it produces	Typical use
Summary	A concise overview of the content, written for high-school students	Quick revision and first reading
Q&A Study Guide	Multiple question-and-answer pairs based on the content	Self-testing and exam preparation
Flashcards	Short cards that turn key facts and terms into easy-to-remember prompts	Memorisation and active recall
Custom Query	An answer to a free-form prompt typed by the user, based on the uploaded content	Clearing doubts and creating tailored material

4.8 Methodology

Development approach

The project was developed in the following stages:

1. **Requirement analysis:** the need for an offline study-material generator was identified and the four output modes were fixed.
2. **Design:** a local RAG architecture was chosen and the application was divided into the interface, ingestion, retrieval and generation modules.
3. **Implementation:** the Streamlit interface, the ingestion pipeline, the ChromaDB integration and the prompt chains were written in Python.
4. **Testing and refinement:** the application was tried with sample educational content and the prompts were refined to improve the quality of the generated material.

Working of the application

The workflow of the application is shown in Figure 4.2 and is explained step by step below.

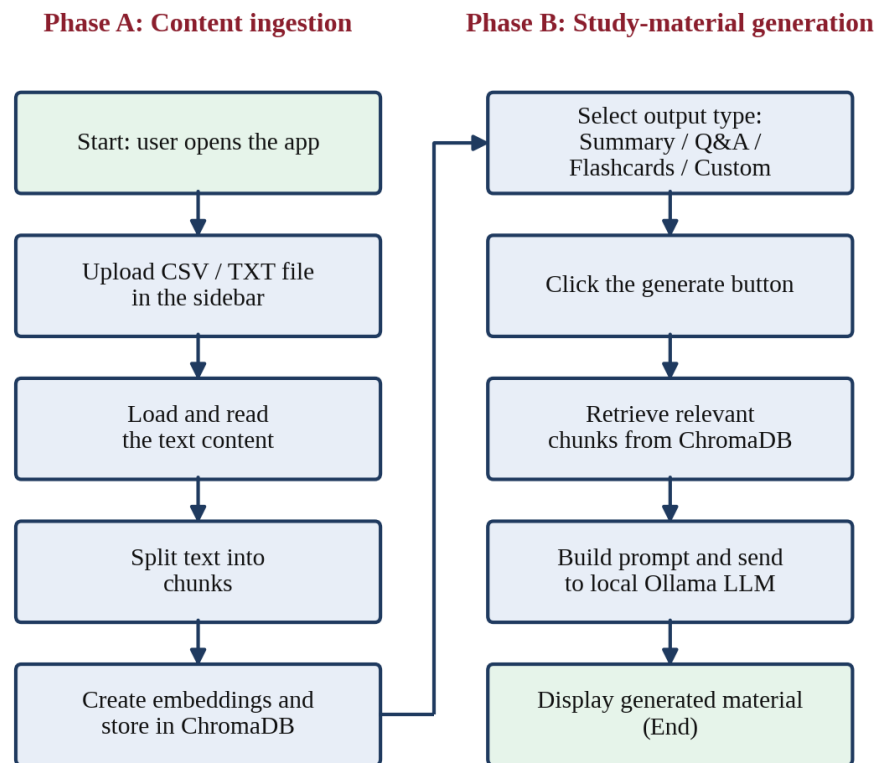


Figure 4.2: Workflow of the application

1. The user opens the application and uploads a CSV or TXT file in the sidebar.
2. The file is loaded and its text content is read.
3. The text is split into chunks so that each piece is small enough to be embedded and retrieved accurately.
4. SentenceTransformers converts every chunk into an embedding, and the embeddings are stored in ChromaDB.
5. The user selects the output type (Summary, Q&A, Flashcards or Custom Query) and clicks the generate button.
6. The retriever finds the chunks most relevant to the request in ChromaDB.
7. A prompt for the selected output type is built with the retrieved chunks and sent to the local LLM through Ollama.
8. The generated study material is displayed in the application.

Prompt design

A separate prompt template is used for each output mode. The summary prompt asks the model to explain the content in language suitable for high-school students. The Q&A prompt asks for several question-and-answer pairs. The flashcard prompt asks for short, memorisable cards. In the custom-query mode the user's own prompt is combined with the retrieved content, which gives flexibility while keeping the answer tied to the uploaded material.

Installation and usage

Ollama must be installed and the server started with `ollama serve`. The application is then set up and run as follows:

1. Install the dependencies:

```
pip install -r requirements.txt
```

2. Run the application:

```
streamlit run app.py
```

3. Upload the CSV or TXT educational content in the sidebar.
4. Select the type of output: Summary, Q&A, Flashcards or Custom Query.
5. Click the button to generate the study material.

The project is open to contributions: issues can be reported, features suggested and pull requests submitted.

Application interface

Figures 4.3 show the user interface of the application.

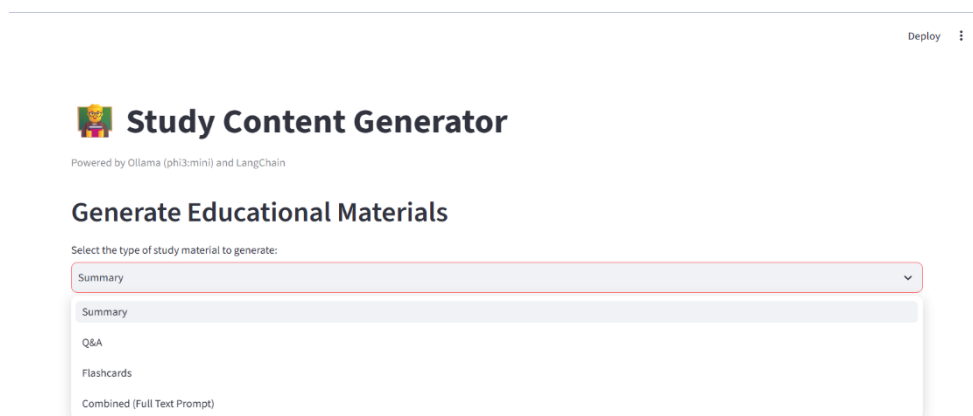


Figure 4.3: User interface of the Study Content Generator

4.9 Expected Outcomes

Project outcomes

- A working web application that turns CSV or TXT educational content into summaries, Q&A study guides, flashcards and custom-query answers.
- A privacy-friendly solution: content stays on the user's computer, and no cloud API key or subscription is needed.
- Quicker preparation of revision material for teachers and students, compared with writing it by hand.
- Answers that stay close to the uploaded material because of vector-based retrieval, which reduces the chance of unrelated output.

Learning outcomes

Through the internship and this project, hands-on experience was gained in Python-based Generative AI development, including prompt design, embeddings, vector databases, the RAG pattern, running LLMs locally with Ollama, orchestrating pipelines with LangChain and building interactive interfaces with Streamlit.

Future scope

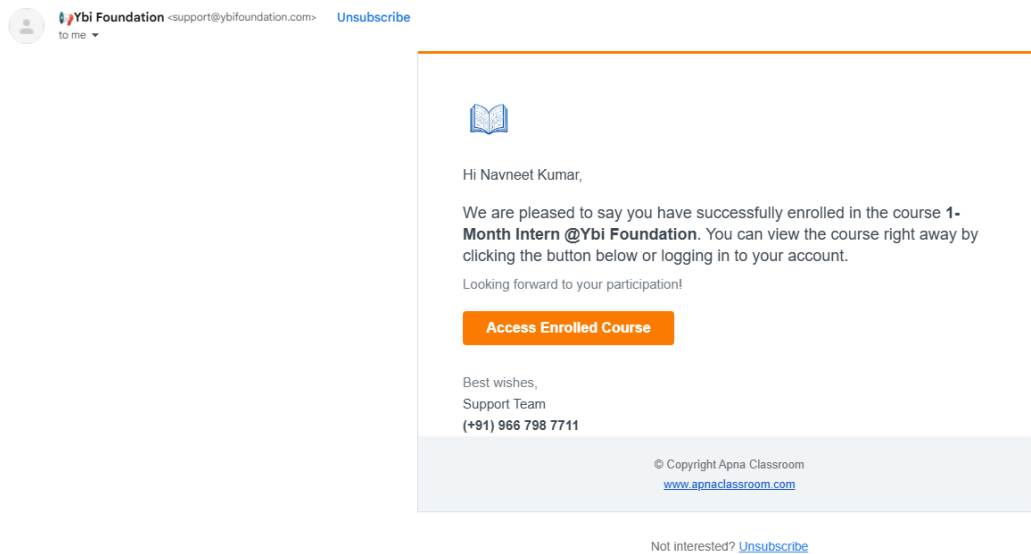
- Support for more input formats such as PDF, DOCX and web pages.
- Options to download or export flashcards and study guides, for example as PDF files.
- A quiz mode with multiple-choice questions and automatic scoring.
- Selection of the Ollama model from within the interface, and support for more languages.
- A systematic evaluation of the quality of the generated material with teachers and students.

Conclusion

Study Content Generator shows how a locally hosted language model, a vector store and a simple web interface can be combined into a practical tool for education. It gives teachers and students a fast way to create study material from their own content while keeping the data private and avoiding the cost of cloud services. The project also gave the author valuable practical exposure to Generative AI during the virtual internship at YBI Foundation.

4.10 Certificates of Completion and Other Communication Mail Proof

The internship completion certificate issued by YBI Foundation and the e-mail communication related to the internship are attached below as proof of completion.



CHAPTER 5: BIBLIOGRAPHY / REFERENCES

- [1] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998-6008.
- [2] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877-1901.
- [3] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982-3992.
- [5] Streamlit Inc., "Streamlit documentation." [Online]. Available: <https://docs.streamlit.io> (accessed Sep. 2026).
- [6] LangChain, "LangChain Python documentation." [Online]. Available: <https://python.langchain.com> (accessed Sep. 2026).
- [7] Ollama, "Ollama: Get up and running with large language models." [Online]. Available: <https://ollama.com> (accessed Sep. 2026).
- [8] Chroma, "Chroma documentation." [Online]. Available: <https://docs.trychroma.com> (accessed Sep. 2026).
- [9] Sentence-Transformers, "SentenceTransformers documentation." [Online]. Available: <https://www.sbert.net> (accessed Sep. 2026).
- [10] Python Software Foundation, "Python 3 documentation." [Online]. Available: <https://docs.python.org/3/> (accessed Sep. 2026).